

AI-Enabled Learning & Competency Gap Platform

Whole-project blueprint for capacity building in India's Official Statistical System

Build target: Antigravity + n8n + API-first backend + RAG/LLM + competency-gap engine + iGOT Karmayogi integration

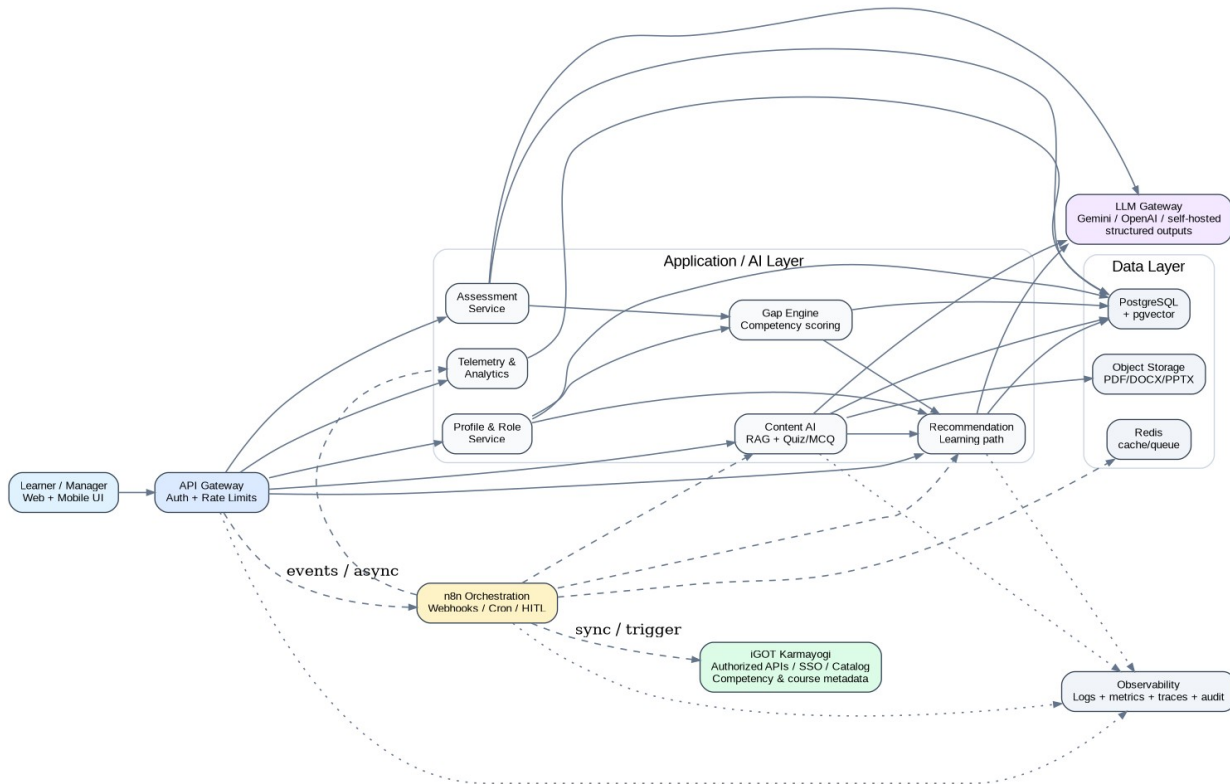


Figure 1. Reference architecture - build this as an intelligent companion layer around iGOT, not as a replacement LMS.

Core principle: keep the learner-facing experience simple, keep the AI outputs grounded and explainable, and keep all integration points behind an adapter layer so official iGOT API contracts can be swapped without redesigning the product.

1. What you are actually building

A role-aware learning intelligence platform with four loops: (1) understand the official and role competency requirements, (2) measure demonstrated competency, (3) recommend the best next learning actions from iGOT and your own content, and (4) continuously re-measure outcomes. The uploaded-material feature adds a fifth loop: turn authoritative learning material into grounded practice questions and quizzes.

Module	Primary job	Output
Learner Profile	Map user -> role -> required competencies	Competency requirement vector
Gap Engine	Compare required vs demonstrated capability	Prioritized skill-gap list
Recommendation Engine	Retrieve/rank relevant iGOT content	Explainable course shortlist
Learning Path	Sequence learning by urgency and prerequisites	Now / Next / Later plan
Content AI	Ingest documents and generate grounded items	RAG index + quiz/MCQ bank
Assessment	Baseline, practice, post-test, re-test	Evidence of mastery
Analytics	Measure progress and system quality	Dashboards + audit trails
Integration Adapter	Connect to iGOT using approved contracts	Stable internal API

Important integration constraint

Do not invent or hard-code iGOT endpoints from the public website. Build an iGOTAdapter interface first, then implement it against the authorized API/SSO/catalog contracts supplied by Karmayogi Bharat/CBC. Official materials indicate that the iGOT ecosystem is competency-linked and supports API-linked applications; the current KCM page also states that KCM is integrated into iGOT and that courses are mapped to competencies. [1] [2]

2. Exact target architecture

Recommended production stack (MVP-first, portable to government-approved infrastructure):

Layer	Choice	Why
Frontend	Next.js / React + Tailwind	Fast admin + learner UI; responsive; easy Antigravity-assisted development
Backend API	Python FastAPI	Strong fit for AI/data workloads; typed APIs; async support
Auth	OIDC/SAML-compatible IdP; production iGOT-approved SSO	Keep auth externalized; avoid duplicate identity stores
Workflow	n8n self-hosted	Orchestrates sync, document jobs, reminders, approvals, and async AI pipelines
Primary DB	PostgreSQL	Users, roles, competencies, assessments, recommendations, audit records
Vector search	pgvector first; OpenSearch/Elasticsearch later if needed	One datastore in MVP; hybrid search can be added
Cache / queue	Redis	Caching, job coordination, rate limiting
Object storage	S3-compatible / government-approved object storage	Original documents, extracted artifacts, question exports
LLM layer	Model gateway + structured JSON outputs	Swap models without changing application logic
Observability	OpenTelemetry + Grafana/Loki or approved equivalents	Trace AI and workflow behavior end-to-end
CI/CD	Git + tests + container build + approved pipeline	Repeatable deployments

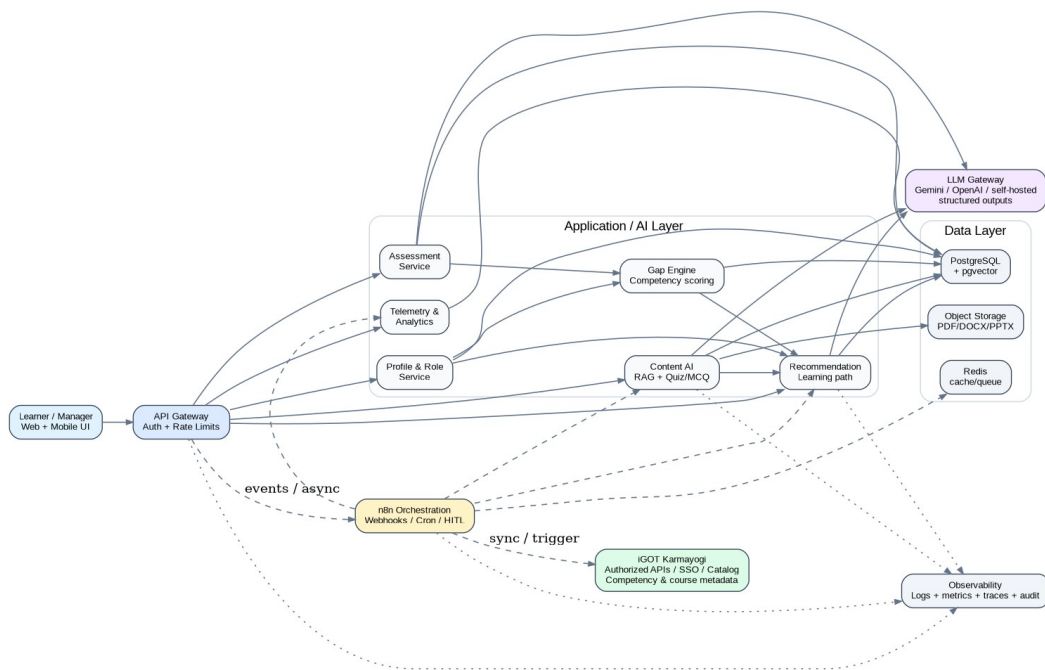


Figure 2. Component architecture and trust boundaries.

Design rule for n8n: use it as the orchestration layer, not as the transaction-processing core. Fast learner requests should hit FastAPI directly. n8n should handle cron/webhook workflows, data sync, async jobs, notifications, document processing, human approvals, and retries. n8n supports visual workflows, code nodes, AI agents, human-in-the-loop controls, and self-hosted/on-prem deployment. [3]

3. Data model - minimum entities

Entity	Key fields	Purpose
User	user_id, org_id, role_id, language, consent_status	Identity/profile
Role	role_id, name, department, level	Job context
Competency	competency_id, type, name, description	Atomic competency taxonomy
RoleCompetency	role_id, competency_id, target_level, weight	Required state
Evidence	user_id, competency_id, source, score, confidence, timestamp	Demonstrated capability
Assessment	assessment_id, competency_ids, mode, rules, version	Test definition
Question	question_id, source_doc, chunk_ref, difficulty, answer, rationale	Assessment item + provenance
LearningResource	resource_id, iGOT_id, title, competencies, duration, language	Course/content catalog
Recommendation	user_id, resource_id, reason, score, model_version	Explainable ranking
LearningPath	path_id, ordered_resources, target_gaps, due_dates	Personalized sequence
TelemetryEvent	event_id, user_id, event_type, payload, timestamp	Behavior/outcome signals
AuditLog	actor, action, object, before/after, timestamp	Traceability

4. End-to-end user workflow



Figure 3. Learner onboarding -> assessment -> recommendation -> continuous re-assessment.

Step	System action	Decision / output
1	Authenticate and capture consent	Create or update user profile
2	Map official role/designation to internal role taxonomy	Select closest role; flag ambiguous mappings
3	Load required competencies	KCM + official-statistics domain competencies
4	Baseline assessment	Question set generated or selected by competency/difficulty
5	Gap analysis	Priority score for each competency
6	Retrieve iGOT resources	Candidates filtered by competency, language, level, status, prerequisites
7	Rank	Explainable score + reason code
8	Build pathway	Now / Next / Later with milestones
9	Practice	Uploaded-material quizzes + MCQs + explanations
10	Telemetry	Completion, score, time, confidence, feedback
11	Re-assessment	Update evidence and close/re-open gaps

5. Recommendation engine - exact MVP logic

Do not start with a black-box recommender. Start with a transparent weighted ranker, log every feature, and collect outcomes. Later, replace only the ranking function.

Feature	MVP weight	Meaning
Competency gap match	0.40	How much the resource addresses high-priority gaps
Difficulty / level fit	0.20	Match to current level and target level
Duration fit	0.15	Fit with learner availability / plan window
Evidence fit	0.10	Matches weak evidence or missing evidence
Language fit	0.10	Preferred language availability
Freshness / quality	0.05	Version, recency, quality signals

Recommendation score = $0.40G + 0.20L + 0.15D + 0.10E + 0.10Lang + 0.05Q$. Hard constraints (access, completion status, prerequisite, validity, approval state) are applied before ranking.

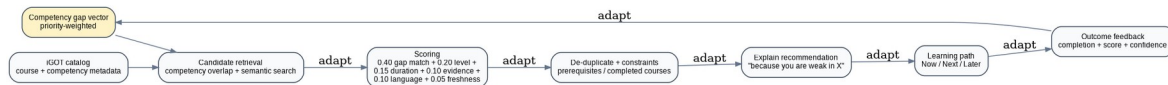


Figure 4. Recommendation loop - retrieve, rank, explain, observe, adapt.

Phase 2: learn a ranking model from outcomes (completion, post-test improvement, gap reduction, learner rating). Never optimize solely for clicks; optimize for competency gain.

6. Uploaded learning material -> Quiz / MCQ pipeline

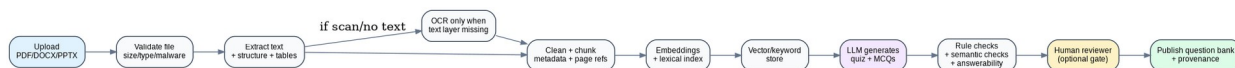


Figure 5. Grounded question-generation pipeline with provenance and human review.

Stage	Implementation detail	Guardrail
Validation	MIME/type, size limit, virus scan, checksum	Reject unsafe/unsupported files
Extraction	PDF parser, DOCX/PPTX parser, table extraction	Preserve page/slide/paragraph references
OCR	Only when text layer absent/poor	Mark OCR confidence
Chunking	Semantic chunks 300-800 tokens, overlap 50-100	Attach source metadata
Indexing	Embeddings + lexical terms	Hybrid retrieval
Question generation	LLM returns strict JSON schema	No free-form persistence
Validation	Answerability, single-best-answer, distractor quality, source coverage	Auto-reject low-confidence items
Review	Human approval for publishable bank	Optional for low-risk practice mode
Publishing	Version question bank + source citation	Immutable provenance

MCQ JSON contract (conceptual): question_id, stem, options[4], correct_option, explanation, competency_id, difficulty, source_document, source_locator, evidence_excerpt, generation_model, prompt_version, validator_scores. Store the source locator even if the learner does not see the raw excerpt.

7. n8n blueprint - workflows to build

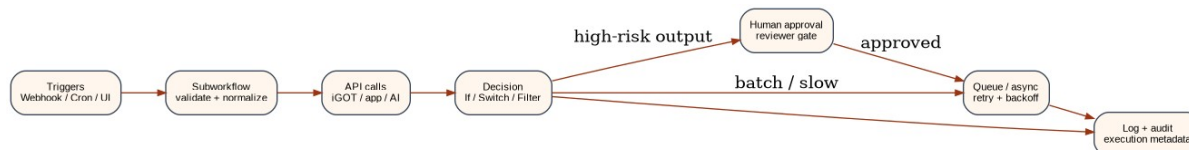


Figure 6. n8n should be the workflow/orchestration layer around your core APIs.

WF	Trigger	Steps	Result
WF-01 Catalog Sync	Cron nightly	Fetch iGOT metadata -> normalize -> diff -> upsert	Current course catalog
WF-02 Profile/Role Sync	Webhook / scheduled	Pull approved profile data -> map role -> update	Fresh role context
WF-03 Baseline Assessment	Webhook	Create assessment -> score -> update evidence -> recompute gaps	Gap vector
WF-04 Recommendation	Webhook / gap change	Retrieve -> rank -> explain -> persist -> notify	Personalized shortlist
WF-05 Document Ingestion	Webhook upload	Validate -> extract -> chunk -> embed -> index	Searchable corpus
WF-06 Quiz Generation	Webhook / queue	Retrieve source chunks -> generate JSON -> validate -> score	Draft quiz bank
WF-07 Human Review	Webhook	Send review -> wait -> publish/reject	Approved items
WF-08 Learning Reminder	Cron	Find due items -> send notification -> log	Engagement
WF-09 Outcome Telemetry	Webhook	Normalize events -> write -> aggregate	Learning analytics
WF-10 Reassessment	Cron / event	Detect due learner -> generate	Updated mastery

		post-test -> compare	
WF-11 Model Evaluation	Cron	Run golden-set tests -> compare metrics -> alert	AI quality report
WF-12 Error Recovery	Error trigger	Retry -> dead-letter -> alert -> audit	Reliable operations

n8n environment strategy

- DEV: local n8n + mock iGOT adapter + sandbox data.
- QA: dedicated n8n + synthetic documents + deterministic mock models where possible.
- PROD: self-hosted n8n with encrypted credentials, RBAC, backups, separate worker capacity when scale requires it.

8. Antigravity development strategy

Use Antigravity as the agentic development environment and engineering control plane; do not make it a production runtime dependency. Google currently describes Antigravity as an agentic development platform with Antigravity IDE, CLI, SDK, and a standalone agent manager; the IDE supports codebase-aware agentic development, Skills, and MCP. [4]

Antigravity task	What to give the agent	Acceptance test
Scaffold	Architecture spec + API contracts + folder structure	App boots, health checks pass
Feature build	One bounded feature + schema + tests	Unit + API tests pass
n8n workflows	Workflow contract + event schema	Workflow runs on fixture payloads
RAG pipeline	Document examples + JSON schemas	Correct source locators + retrieval tests
UI	Wireframe + design tokens + role behavior	Playwright smoke tests
Security	Threat model + auth rules	Unauthorized access tests fail safely
Refactor	Measured problem + constraints	No regression in test suite

Recommended repo structure: /apps/web, /services/api, /services/ai, /services/recommender, /services/ingestion, /packages/schemas, /packages/prompts, /infra, /n8n/workflows, /tests, /docs. Commit n8n workflow JSON exports and version prompts/schemas like code.

9. What you need to learn - complete roadmap

Difficulty scale: B = Beginner, I = Intermediate, A = Advanced, E = Expert. Priority P0 = must know for MVP, P1 = needed soon, P2 = later optimization.

9A. Domain - Official Statistics

Topic	Difficulty	Priority
Indian Official Statistical System	B	P0
MoSPI / NSO institutional landscape	B	P0
Statistical standards and classifications	I	P0
Official statistics lifecycle	B	P0
Census and administrative data concepts	B	P0
Sample surveys	I	P0
Sampling frames and non-response	I	P0
Estimation and weighting	I	P0
Index numbers	I	P1
National accounts basics	I	P1
Economic statistics basics	I	P1
Social statistics basics	I	P1

Labour statistics basics	I	P1
Agricultural statistics basics	I	P2
Price statistics / CPI concepts	I	P1
Statistical disclosure control	A	P1
Confidentiality and microdata handling	A	P0
Metadata and data dictionaries	I	P0
Data quality dimensions	I	P0
Dissemination and statistical communication	I	P0
GIS/spatial statistics concepts	A	P2
Indian government data portals / APIs	I	P1

9B. Software engineering

Topic	Difficulty	Priority
Python fundamentals	B	P0
Python virtual environments / packaging	B	P0
Git / GitHub workflows	B	P0
Linux CLI basics	B	P0
HTTP fundamentals	B	P0
REST API design	I	P0
JSON / schema validation	B	P0
FastAPI	I	P0
SQL fundamentals	B	P0
PostgreSQL	I	P0
Database indexing	I	P1
Transactions / isolation	A	P1
ORM / SQLAlchemy	I	P0
Unit testing / pytest	I	P0
API integration testing	I	P0
Playwright/browser testing	I	P1
Docker	I	P0
CI/CD basics	I	P1
Code quality / linting	I	P1
Async programming	A	P1

9C. Data science and ML

Topic	Difficulty	Priority
NumPy	B	P0
Pandas	B	P0
Data cleaning	B	P0
Exploratory data analysis	B	P0
Probability	I	P0
Descriptive statistics	B	P0
Inferential statistics	I	P0
Regression	I	P1
Classification	I	P1
Clustering	I	P2
Feature engineering	I	P1
Cross-validation	I	P1
Model metrics	I	P1
Bias/variance	A	P1
Calibration	A	P2
Recommender systems	A	P1

Learning-to-rank	A	P2
Experiment design / A-B testing	A	P1
Model monitoring	A	P1

9D. GenAI / RAG / NLP

Topic	Difficulty	Priority
LLM fundamentals	B	P0
Prompt engineering	B	P0
Structured output / JSON schema	I	P0
Embeddings	I	P0
Chunking strategies	I	P0
Vector search	I	P0
Hybrid search	I	P1
RAG architecture	I	P0
RAG evaluation	A	P0
Grounding and citations	I	P0
Hallucination mitigation	A	P0
Context management	A	P1
Reranking	A	P2
LLM tool calling	I	P1
Agentic workflows	A	P1
Model routing	A	P2
Guardrails	A	P0
Prompt/version management	I	P0
Golden datasets / eval harnesses	A	P0
Cost/latency optimization	A	P1

9E. Document AI + quiz generation

Topic	Difficulty	Priority
PDF text extraction	I	P0
DOCX parsing	I	P0
PPTX parsing	I	P0
Table extraction	A	P1
OCR fundamentals	I	P0
OCR confidence	A	P1
Document chunk provenance	I	P0
Semantic question generation	I	P0
MCQ distractor design	I	P0
Answerability checks	A	P0
Question difficulty calibration	A	P1
Bloom taxonomy	I	P0
Question duplication detection	I	P1
Human review workflows	I	P0

9F. Assessment science

Topic	Difficulty	Priority
Assessment blueprinting	I	P0
Item writing principles	I	P0
Bloom taxonomy	I	P0
Classical Test Theory	A	P1
Item difficulty / discrimination	A	P1

Reliability / validity	A	P1
IRT fundamentals	E	P2
Adaptive testing concepts	A	P2
Question bank assembly	I	P0
Randomization	I	P0
Scoring rules	I	P0
Cheating / integrity controls	A	P1

9G. Architecture / DevOps / security

Topic	Difficulty	Priority
Layered architecture	B	P0
Microservices trade-offs	A	P1
Event-driven architecture	A	P1
Caching	I	P1
Queues / retries / dead letters	I	P0
Redis	I	P0
Rate limiting	I	P1
API gateways	I	P1
Secrets management	I	P0
RBAC	I	P0
SSO/OIDC/SAML concepts	I	P0
Encryption in transit / at rest	I	P0
Threat modeling	A	P0
OWASP API security	A	P0
Audit logging	I	P0
Observability	I	P0
OpenTelemetry	A	P1
Backup / recovery	I	P1
High availability	A	P2
Vulnerability management	A	P1

9H. n8n

Topic	Difficulty	Priority
Triggers and webhooks	B	P0
HTTP Request node	B	P0
Expressions and data mapping	B	P0
If / Switch / Merge	B	P0
Loops / batches	I	P0
Sub-workflows	I	P0
Credentials / secrets	I	P0
Error workflows	I	P0
Retry / backoff patterns	I	P0
Human-in-the-loop	I	P0
Code nodes (JS/Python)	I	P1
Execution logs / observability	I	P0
Workflow versioning	I	P1
Queue/scaling concepts	A	P1
Webhook security	I	P1

9I. Antigravity

Topic	Difficulty	Priority
-------	------------	----------

Agentic IDE basics	B	P0
Codebase context / instructions	B	P0
Agent tasks and bounded prompts	B	P0
Skills	I	P0
MCP concepts	I	P1
Agent-driven test generation	I	P0
Browser automation with Playwright	I	P1
Agent review loops	I	P0
CLI workflows	I	P1
SDK concepts	A	P2
Human approval for high-impact changes	I	P0

10. Learning sequence - do not learn everything at once

Week	Focus	Deliverable
1	Python + Git + HTTP + JSON	Hello API + Git repo
2	SQL + PostgreSQL + FastAPI	CRUD service + migrations
3	Docker + tests + auth basics	Containerized API + test suite
4	n8n fundamentals	3 working workflows + webhooks
5	Official statistics domain	Competency taxonomy v1
6	Statistics + assessment basics	Baseline assessment engine
7	LLM + embeddings + chunking	RAG prototype
8	Document AI	PDF/DOCX/PPTX ingestion
9	MCQ generation + validation	Question bank MVP
10	Recommendation engine	Gap -> ranked resources
11	iGOT adapter	Mock adapter + authorized API contract mapping
12	Analytics + audit + security	Production readiness baseline
13-14	Pilot + evaluation	10-30 user pilot + error analysis
15-16	Hardening	Scale, security, UX, governance

11. Step-by-step build plan from zero

Phase	What to build
Phase 0 - Freeze scope	Define MVP users, 5-10 roles, 30-50 competencies, one language, one document format family, 50-100 iGOT resources.
Phase 1 - Repo + environments	Create monorepo, Docker, PostgreSQL, Redis, FastAPI, Next.js, n8n, CI checks.
Phase 2 - Competency model	Create role, competency, target-level, weight and prerequisite tables; seed initial official-statistics competency map.
Phase 3 - Profile + assessment	Implement auth adapter, profile screen, question bank schema, assessment runner and evidence scoring.
Phase 4 - Gap engine	Implement transparent gap formula, prioritization, confidence and explainability.
Phase 5 - iGOT adapter	Implement catalog, competency, user/progress connectors only from approved API contracts; use mocks until access is granted.
Phase 6 - Recommendation	Retrieve candidates, apply hard filters, rank with weighted score, generate explanations, persist versioned recommendations.
Phase 7 - Document AI	Build secure upload, parse/OCR, chunk, embed, index, provenance.
Phase 8 - Quiz AI	Generate structured MCQs, run deterministic + LLM validators, review, publish.

Phase 9 - n8n	Move sync, async processing, reminders, review routing and telemetry aggregation into n8n.
Phase 10 - Analytics	Build learner, manager and admin dashboards; monitor gap closure and AI quality.
Phase 11 - Security/governance	Threat model, RBAC, data classification, retention, audit, model usage policy, human review gates.
Phase 12 - Pilot	Run controlled pilot; compare baseline vs post-test, recommendation acceptance, completion and gap reduction.
Phase 13 - Scale	Add hybrid search, better ranking, queue workers, event streaming only when metrics justify them.

12. Testing & acceptance criteria

Area	Minimum acceptance test
Competency mapping	Every role maps to competencies with target levels and weights; ambiguous mappings are reviewable.
Gap engine	Same inputs always produce same baseline score; explanation references exact competencies/evidence.
Recommendation	No recommendation violates hard constraints; top results visibly match priority gaps.
RAG	Every generated answer/question has a source locator; unsupported claims are rejected or flagged.
MCQ quality	Exactly one correct answer; distractors plausible; answer is supported by source chunk; duplicate rate below threshold.
iGOT integration	Connector retries safely, handles timeouts, logs request IDs, and can be switched to mock mode.
n8n	Failed jobs retry; dead-letter path exists; manual rerun is possible.
Security	Unauthorized user cannot read another user's assessments/documents; secrets never appear in logs.
Observability	Trace from upload -> AI generation -> validation -> publish exists.
AI evaluation	Golden set tracks groundedness, answerability, difficulty agreement and regression over time.

13. MVP scope and what NOT to build initially

Recommended MVP: one web app, one backend, one Postgres/pgvector instance, one Redis, one n8n instance, one model gateway, 5-10 official-statistics roles, 30-50 competencies, and a small curated iGOT course set. Keep the system modular but physically simple.

Do NOT start with: Kafka, microservices everywhere, custom deep-learning recommenders, IRT/adaptive testing, multi-agent swarms, mobile apps, full LMS replacement, or scraping the iGOT UI. Add each only after a measured need.

Security baseline

- Treat uploaded documents and learner telemetry as controlled data. Apply classification, retention and access rules before connecting production data.
- Keep model prompts/outputs out of generic application logs when they may contain sensitive content; store governed audit references instead.
- Separate document storage, vector indexes and metadata access by tenant/department if required.
- Put a human gate in front of publishable assessment items when correctness matters.

14. One-page blueprint

USER -> AUTH -> ROLE/COMPETENCY MAP -> BASELINE ASSESSMENT -> GAP VECTOR -> iGOT CATALOG
RETRIEVAL -> RANKING -> LEARNING PATH -> LEARN/PRACTICE -> TELEMETRY -> RE-ASSESS -> GAP
VECTOR

UPLOAD MATERIAL -> VALIDATE -> EXTRACT/OCR -> CHUNK + PROVENANCE -> EMBED/HYBRID INDEX -> RAG
RETRIEVE -> GENERATE MCQs -> VALIDATE -> HUMAN REVIEW -> PUBLISH -> PRACTICE -> OUTCOME
TELEMETRY

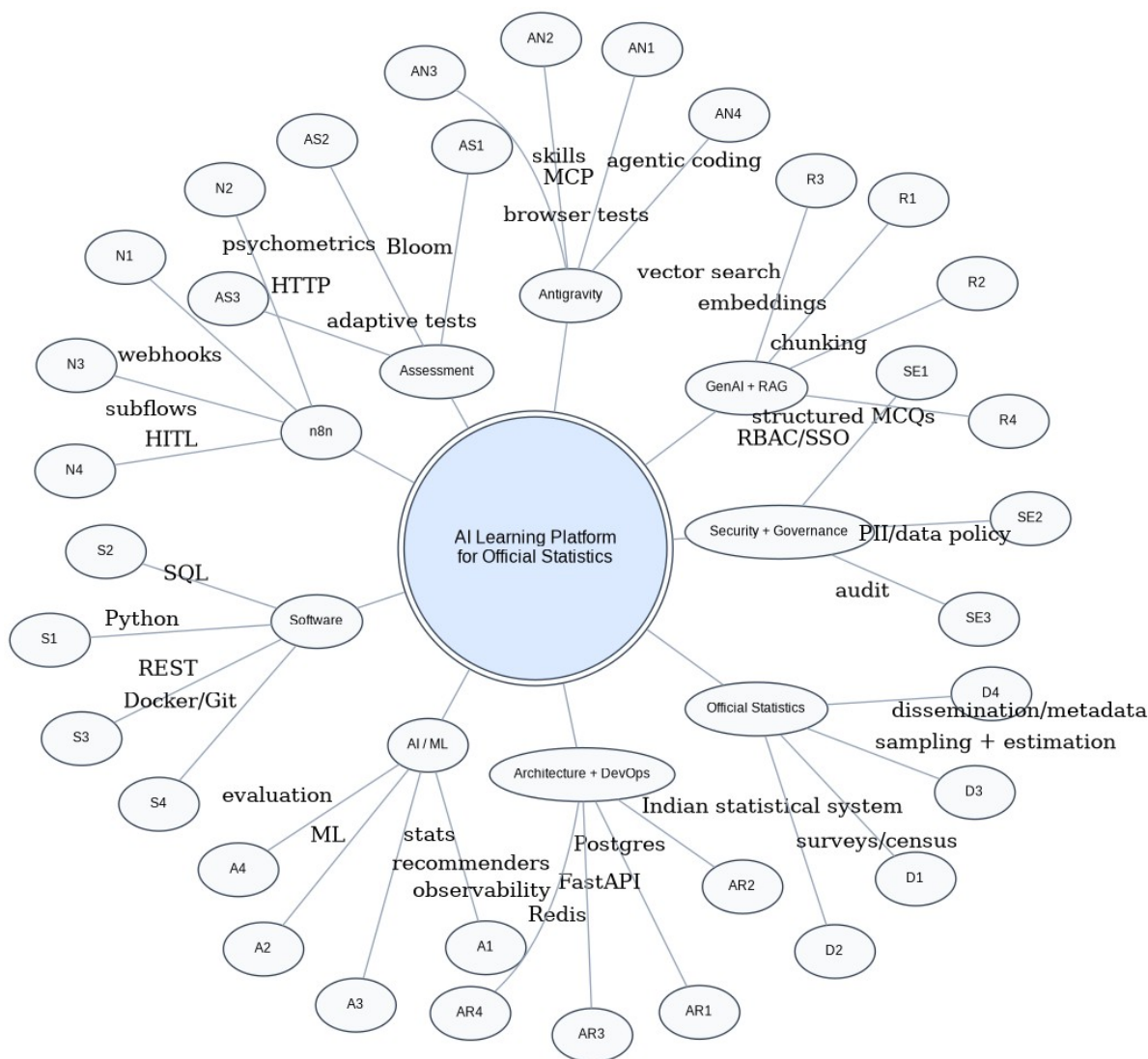


Figure 7. Learning map - the skill domains you need to master to build the product yourself.

The fastest route is to become strong in Python/FastAPI/PostgreSQL, n8n, RAG/document AI, assessment design, and the official-statistics domain first. Recommendation learning can begin with a transparent scoring engine rather than advanced ML.

15. Sources and current integration notes

[1] Capacity Building Commission - Karmayogi Competency Model (KCM), current page states that KCM defines required competencies, courses on iGOT are mapped to KCM, and KCM is integrated into iGOT.

<https://www.cbc.gov.in/karmayogi-competency-model-kcm>

[2] Capacity Building Commission procurement clarification (2026) - states that iGOT will be used as LMS and an application will be linked to iGOT through APIs, with transition to CBC/KB infrastructure after the project period. <https://cbc.gov.in/sites/default/files/2026-01/Ammends-1.pdf>

[3] n8n official AI/workflow product pages - visual workflows, AI agents, human-in-the-loop, self-hosting/on-prem deployment, code nodes and integrations. <https://n8n.io/ai/> and <https://n8n.io/>

[4] Google Antigravity official codelabs - current Antigravity platform, Antigravity IDE, CLI, SDK, Skills and MCP. <https://codelabs.developers.google.com/getting-started-agy-ide>

[5] iGOT Karmayogi official portal - current platform and learning ecosystem. <https://www.igotkarmayogi.gov.in/>

Version note: this blueprint intentionally avoids hard-coded iGOT endpoint names because API contracts, authentication methods, scopes and payloads should be taken from the authorized integration documentation provided to your project. The architecture therefore uses an internal adapter with a mock implementation first.